Name: Prasanna V

Reg no: 19244260339

CSA0609- Design and Analysis of Algorithms

BINARY SEARCH WITH BIG-O ANALYSIS

AIM

To implement the **Binary Search algorithm** in Python and analyze its **time and space complexity**.

ALGORITHM

1. Start.
2. Take a **sorted array**.
3. Set low = 0 and high = n - 1.
4. Find the middle element.
5. If the middle element equals the search element, return its position.
6. If the search element is greater, search the right half.
7. Otherwise, search the left half.
8. Repeat until the element is found or the search range becomes empty.
9. Stop.

PSEUDOCODE

```
BinarySearch(A, key)
low = 0
high = n - 1
while low <= high
  mid = (low + high) / 2
  if A[mid] == key
    return mid
  else if A[mid] < key
    low = mid + 1
  else
    high = mid - 1
  end if
end while
```

return -1

PYTHON CODE

main.py Visualize Run ▶

```python
def binary_search(A, key):
    low, high = 0, len(A) - 1

    while low <= high:
        mid = (low + high) // 2

        if A[mid] == key:
            return mid
        elif A[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1


A = [10, 20, 30, 40, 50]
key = 40

pos = binary_search(A, key)

if pos != -1:
    print("Element found at position", pos + 1)
else:
    print("Element not found")
```

OUTPUT

Output

```
Element found at position 4
```

COMPLEXITY ANALYSIS

Let n be the number of elements.

- **Best Case:** O(1)
- **Average Case:** O(log n)
- **Worst Case:** O(log n)
- **Space Complexity:** O(1)

Big-O Time Complexity: O(log n)

RESULT

Thus, the Binary Search algorithm was successfully implemented in Python, and its time complexity was analyzed as O(log n) with O(1) space complexity.